it's really tricky to manage the namespaces and hard to keep the dummy and the real user in sync.

- You can have some other object in the system filter the necessary information to CreatesProject. In the case of access information, the controller is a logical place for that; it's arguably the controller's role to police access to other services. In other cases, whoever invokes the CreatesProject action might be responsible for sending the necessary data as simple Ruby data, like a hash or even a simple struct.
- Similarly, you can create a wrapper object—ProjectCreatingUser or something—that wraps a user and provides an API for CreatesProject. In tests, the ProjectCreatingUser can easily stub information, making the dependency a single plain Ruby object rather than a complex tree of ActiveRecord objects.

The last two options may seem like way, way too much structure to you. (Maybe you came to Rails to avoid the kind of hyper-indirect object models that characterize, say, enterprise Java projects.) And sometimes I'd agree. In most cases I wouldn't start with this level of complexity and indirection. But I would refactor to it when things start to get complex—this kind of refactoring is much easier to do if you catch it early.

The point is that writing application tests that bypass Rails requires you to be more thoughtful about your dependencies (or, looked at from the other direction, writing tests that use Rails autoloading allows you to be sloppy about dependencies). However you choose to manage the dependencies, being aware of them is the first step in keeping your program easy to change over time.

Prescription 52	You don't need to start with elaborate object indirection, but it's useful when logic gets complicated.

Running Rails-Free Tests

If you're going to have some tests that run without loading the Rails environment, you need to be careful to run those tests separately from the slower, Rails-loading tests. You lose the benefit of not loading Rails if some of your tests *load Rails*. Also, if you're doing any namespace munging in your fast tests, mixing and matching fast and slow tests is guaranteed to give weird errors.

The easiest way to manage running fast tests is to keep them in their own directory or directories and pass those directories as arguments to ruby. This is easy in RSpec, since the spec command will take directories or globs as arguments:

```
$ rspec spec/models spec/workflows
```